

Deploying a Spring Boot backend involves packaging the application and running it on a server or cloud platform. Spring Boot applications can be packaged as self-contained executable JAR files, which simplifies deployment because they include an embedded web server (like Tomcat, Jetty, or Undertow).

Common Deployment Strategies

You have several options for deployment, depending on your needs for control, scalability, and ease of use:

- **Platform-as-a-Service (PaaS) Providers:** These platforms abstract away infrastructure management, allowing you to focus on your code. They often use buildpacks to automatically detect and run your Spring Boot app from a JAR file or source code repository.
 - **Examples:** Heroku, Railway, Render, Microsoft Azure Spring Apps, AWS Elastic Beanstalk.
- **Containerization (Docker):** This approach packages your application and its dependencies into a consistent, portable container. You build a Docker image, push it to a registry (like Docker Hub), and deploy it to a container orchestration platform.
 - **Examples:** Deploying to Kubernetes (AKS, EKS, etc.) or container-focused PaaS like Render.
- **Virtual Private Servers (VPS) / Infrastructure-as-a-Service (IaaS):** This provides more control, requiring manual setup of the server environment (e.g., Linux OS, Java runtime, Nginx as a reverse proxy).
 - **Examples:** AWS EC2, DigitalOcean, Linode.
- **Traditional Server Deployment:** For legacy or specific enterprise environments (like CICS), the application can be packaged as a WAR file and deployed to a traditional application server.

General Steps for PaaS/VPS Deployment

The most common process involves these general steps:

1. **Package your application:** Use [Maven](#) or Gradle to build an executable JAR file.
 1. For Maven, run `mvn clean package` in your project's root directory. The JAR file will be in the `/target` directory.
 2. For Gradle, run `gradle build`. The JAR will be in the `/build/libs` directory.

2. **Prepare the target environment:** Configure necessary environment variables for things like database credentials, API keys, or active profiles (e.g., `SPRING_PROFILES_ACTIVE=production`).

3. **Upload and deploy:**

1. **PaaS:** Link your GitHub repository to the service or use their CLI tools to deploy the artifact. The platform automatically handles running the `java -jar` command.
2. **VPS/IaaS:** Upload the JAR file to your server (e.g., via SCP or an FTP tool) and run it using the `java -jar your-application.jar` command. Consider using a process manager like `systemctl` or `supervisor` to ensure it runs as a service and restarts automatically.